

```
In [103... import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error

# Load the data
file_path = '/Users/nickfolino/Documents/Coding Projects/Employee_Metric_Synthetic.csv'
data = pd.read_csv(file_path)

# Remove columns not relevant for feature importance calculation
excluded_columns = ['flight_risk', 'Emp_ID', 'risk_rank', 'employee_id', 'risk_score']
feature_data = data.drop(columns=excluded_columns)

# Separate features and target variable
X = feature_data.drop(columns=['tenure'])
y = feature_data['tenure']

# Encode categorical variables
X = pd.get_dummies(X, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Random Forest model
model = RandomForestRegressor(random_state=42, n_estimators=100)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

# Feature importance
feature_importances = pd.DataFrame({
    'Feature': X.columns,
```

```
'Importance': model.feature_importances_
}).sort_values(by='Importance', ascending=False)

r2, mae, feature_importances
```

```
Out[103...] (0.7450917194276916,
4.8454119282328465,
           Feature  Importance
1    prod_change    0.650895
2    abs_change    0.251157
4    comp_market    0.028095
5  incentive_pay    0.027003
0      eng_score    0.024328
3    satisfaction    0.018523)
```

```
In [105...] # Extract feature importance values for weighting
feature_weights = feature_importances.set_index('Feature')['Importance']

# Adjust the weights based on the direction of their relationship to tenure
# Negative relationship = higher values/increased risk; Positive relationship = higher values/decreased risk
weight_adjustments = {
    'satisfaction': -1, # Lower satisfaction increases risk
    'flight_risk': 0,   # Flight risk is neutral in this context
    'incentive_pay': -1, # Higher incentive pay decreases risk
    'prod_change': -1,  # Higher productivity change decreases risk
    'abs_change': 1,    # Higher absenteeism change increases risk
    'comp_market': 1,   # Assuming higher market comp increases risk
    'eng_score': 0      # Eng_score is irrelevant (neutral weight)
}

# Adjust feature weights based on their importance and direction
adjusted_weights = feature_weights * pd.Series(weight_adjustments)

# Normalize the weights to sum to 1 for consistent scaling
adjusted_weights = adjusted_weights / adjusted_weights.abs().sum()

# Rescale each variable to a score between 1 and 100
current_data = X.copy() # Using current X data (independent variables)
scaled_data = current_data.copy()
for column in current_data.columns:
    column_min = current_data[column].min()
    column_max = current_data[column].max()
```

```

scaled_data[column] = 100 * (current_data[column] - column_min) / (column_max - column_min)

# Ensure all risk scores are positive by adding an offset to make the minimum score 1
scaled_data = scaled_data + 1

# Apply adjusted weights to scaled scores and calculate the weighted risk score for each employee
risk_scores = scaled_data.mul(adjusted_weights.abs(), axis=1).sum(axis=1)

# Add risk scores to the dataset and rank employees by their scores
employee_risk = data.copy()
employee_risk['risk_score'] = risk_scores
employee_risk['risk_rank'] = employee_risk['risk_score'].rank(ascending=False)

# Sort employees by risk rank
employee_risk_sorted = employee_risk.sort_values(by='risk_rank', ascending=True)
employee_risk_sorted[['risk_rank', 'prod_change', 'satisfaction', 'abs_change', 'comp_market',
                      'incentive_pay', 'risk_score']].head()

```

Out[105]...

	risk_rank	prod_change	satisfaction	abs_change	comp_market	incentive_pay	risk_score
52	1.0	93.949894	16.426580	85.113667	63.935607	15851.301705	89.275383
183	2.0	87.733935	45.328883	87.747201	148.408347	16963.386510	88.962432
178	3.0	93.672999	3.654967	81.446848	126.718829	8007.870901	88.256005
80	4.0	86.310343	89.389258	89.000534	66.628337	18077.652916	86.986568
11	5.0	96.990985	18.704375	69.189520	74.317219	8475.075521	86.155739

In [105]...

```

# Extract feature importance values for weighting
feature_weights = feature_importances.set_index('Feature')['Importance']

# Adjust the weights based on the direction of their relationship to tenure
# Negative relationship = higher values/increased risk; Positive relationship = higher values/decreased risk
weight_adjustments = {
    'satisfaction': -1, # Lower satisfaction increases risk
    'flight_risk': 0,   # Flight risk is neutral in this context
    'incentive_pay': -1, # Higher incentive pay decreases risk
    'prod_change': -1,  # Higher productivity change decreases risk
    'abs_change': 1,    # Higher absenteeism change increases risk
    'comp_market': 1,   # Assuming higher market comp increases risk
}

```

```
'eng_score': 0      # Eng_score is irrelevant (neutral weight)
}

# Adjust feature weights based on their importance and direction
adjusted_weights = feature_weights * pd.Series(weight_adjustments)

# Normalize the weights to sum to 1 for consistent scaling
adjusted_weights = adjusted_weights / adjusted_weights.abs().sum()

# Rescale each variable to a score between 1 and 100
current_data = X.copy() # Using current X data (independent variables)
scaled_data = current_data.copy()
for column in current_data.columns:
    column_min = current_data[column].min()
    column_max = current_data[column].max()
    scaled_data[column] = 100 * (current_data[column] - column_min) / (column_max - column_min)

# Ensure all risk scores are positive by adding an offset to make the minimum score 1
scaled_data = scaled_data + 1

# Apply adjusted weights to scaled scores and calculate the weighted risk score for each employee
risk_scores = scaled_data.mul(adjusted_weights.abs(), axis=1).sum(axis=1)

# Add risk scores to the dataset and rank employees by their scores
employee_risk = data.copy()
employee_risk['risk_score'] = risk_scores
employee_risk['risk_rank'] = employee_risk['risk_score'].rank(ascending=False)

# Sort employees by risk rank
employee_risk_sorted = employee_risk.sort_values(by='risk_rank', ascending=True)
employee_risk_sorted[['risk_rank', 'prod_change', 'satisfaction', 'abs_change', 'comp_market',
                    'incentive_pay', 'risk_score']].head()
```

Out [105...

	risk_rank	prod_change	satisfaction	abs_change	comp_market	incentive_pay	risk_score
52	1.0	93.949894	16.426580	85.113667	63.935607	15851.301705	89.275383
183	2.0	87.733935	45.328883	87.747201	148.408347	16963.386510	88.962432
178	3.0	93.672999	3.654967	81.446848	126.718829	8007.870901	88.256005
80	4.0	86.310343	89.389258	89.000534	66.628337	18077.652916	86.986568
11	5.0	96.990985	18.704375	69.189520	74.317219	8475.075521	86.155739

In [107...

```

import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders

# Save the sorted employee risk dataset to a CSV file
output_file_path = '/Users/nickfolino/Documents/Coding Projects/employee_risk_scores.csv'
employee_risk_sorted.to_csv(output_file_path, index=False)

# Extract the Emp_IDs of the top 5 ranked employees
top_5_employees = employee_risk_sorted.head(5)
top_5_emp_ids = top_5_employees['Emp_ID'].tolist()

# Email details
sender_email = "me@mail.co" # Replace with your Gmail address
receiver_email = "you@mail.co" # Replace with the recipient's email address
password = "Password123" # Replace with your Gmail app password

# Email subject and body
subject = "Top Employee Risk Scores"
body = (
    f"Hello,\n\n"
    f"Attached is the dataset containing employee risk scores. Below are the Emp_IDs of the top 5 employees\n\n"
    f"ranked by their risk score:\n\n"
    f"1. {top_5_emp_ids[0]}\n\n"
    f"2. {top_5_emp_ids[1]}\n\n"
    f"3. {top_5_emp_ids[2]}\n\n"
    f"4. {top_5_emp_ids[3]}\n\n"
    f"5. {top_5_emp_ids[4]}\n\n"

```

```

        f"Best regards,"
    )

    # Create the email
    message = MIMEMultipart()
    message["From"] = sender_email
    message["To"] = receiver_email
    message["Subject"] = subject

    # Attach the body text
    message.attach(MIMEText(body, "plain"))

    # Attach the CSV file
    with open(output_file_path, "rb") as attachment:
        part = MIMEBase("application", "octet-stream")
        part.set_payload(attachment.read())
    encoders.encode_base64(part)
    part.add_header(
        "Content-Disposition",
        f"attachment; filename=employee_risk_scores.csv",
    )
    message.attach(part)

    # Send the email
    try:
        with smtplib.SMTP("smtp.gmail.com", 587) as server:
            server.starttls()
            server.login(sender_email, password)
            server.sendmail(sender_email, receiver_email, message.as_string())
            print("Email sent successfully!")
    except Exception as e:
        print(f"Failed to send email: {e}")

    output_file_path, top_5_emp_ids

```

Failed to send email: (534, b'5.7.9 Application-specific password required. For more information, go to\n5.7.9 <https://support.google.com/mail/?p=InvalidSecondFactor> 3f1490d57ef6-e57d15f4e0bsm1050127276.2 - gsmt p')

```

Out[107...] ('/Users/nickfolino/Documents/Coding Projects/employee_risk_scores.csv',
             [53, 84, 79, 81, 12])

```

```
In [111... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error

# Visualization functions
def plot_feature_importances():
    plt.figure(figsize=(10, 6))
    sns.barplot(x='Importance', y='Feature', data=feature_importances)
    plt.title('Feature Weights')
    plt.xlabel('Importance')
    plt.ylabel('Feature')
    plt.tight_layout()
    plt.show()

def plot_residuals():
    residuals = y_test - y_pred
    plt.figure(figsize=(10, 6))
    sns.histplot(residuals, kde=True, color='purple')
    plt.title('Residuals Distribution')
    plt.xlabel('Residuals')
    plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()

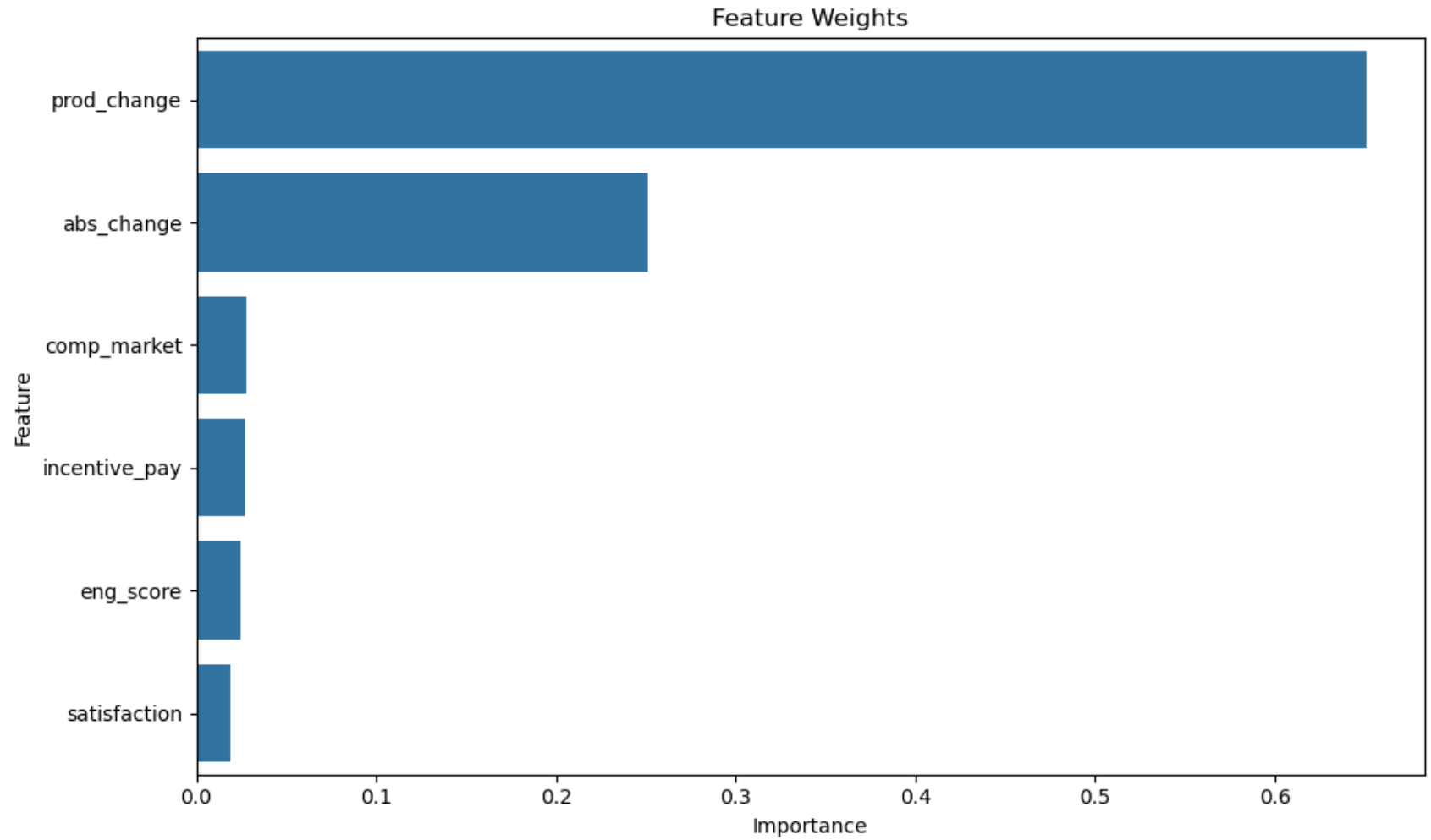
def plot_predictions():
    plt.figure(figsize=(10, 6))
    plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
    plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2)
    plt.title('Actual vs Predicted Tenure')
    plt.xlabel('Actual Tenure')
    plt.ylabel('Predicted Tenure')
    plt.tight_layout()
    plt.show()

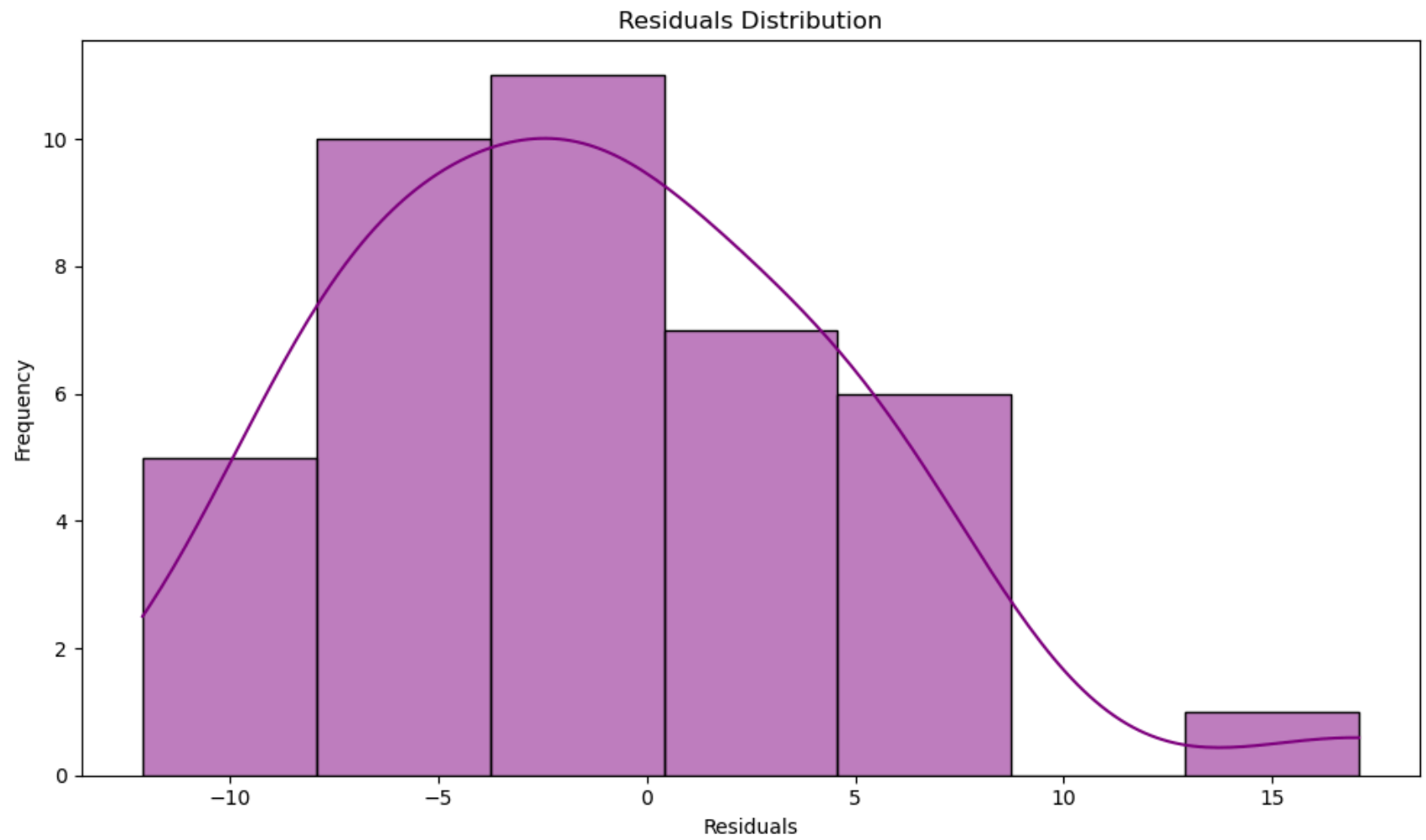
# Display visualizations
print(f"Model R^2: {r2:.2f}")
print(f"Mean Absolute Error: {mae:.2f}")
```

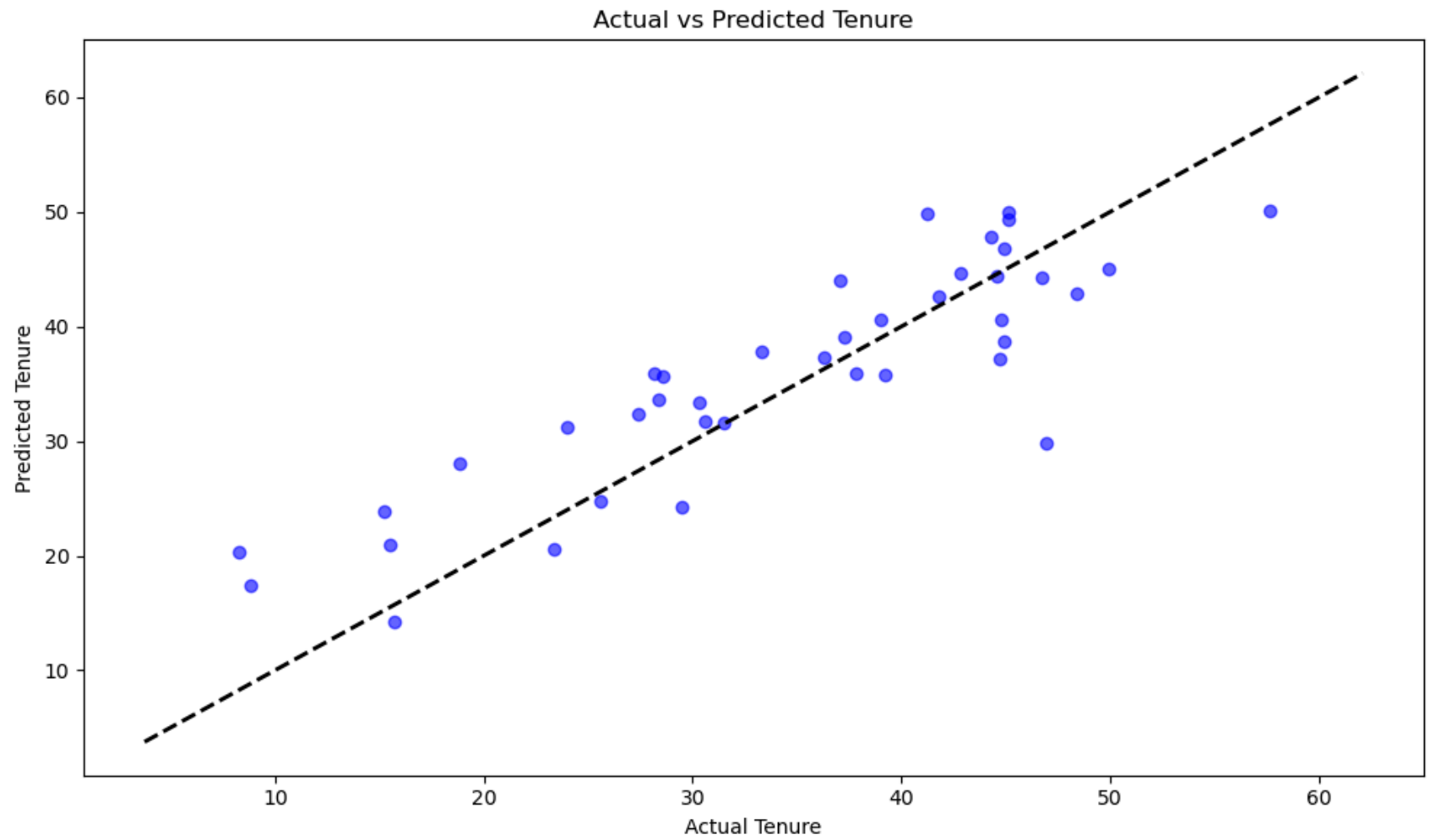
```
plot_feature_importances()  
plot_residuals()  
plot_predictions()
```

Model R^2 : 0.75

Mean Absolute Error: 4.85







In []: